Gate-Model Quantum Neighborhood Search for the Permutation Flow Shop Problem via Fixed-Angle QAOA

Remigiusz Wojewódzki* and Wojciech Bożejko

Wrocław University of Science and Technology, Faculty of Information and Communication Technology, Department of Control and Quantum Computing, Wrocław, Poland,
remigiusz.wojewodzki@pwr.edu.pl

Abstract. The permutation flow shop problem (PFSP) is an NP-hard scheduling problem, and local-search solvers for it spend most of their time evaluating neighborhood moves. Earlier work moved this evaluation to quantum annealers by writing each neighborhood as a quadratic unconstrained binary optimization (QUBO) problem. This paper asks whether the same QUBOs can be solved on gate-model processors. We embed the Quantum Approximate Optimization Algorithm (QAOA) in iterated local search (ILS) and simulated annealing (SA), reuse the annealing QUBOs of four neighborhoods (Adjacent, Fibonacci, Dynasearch, Motzkin), and fix the QAOA angles offline, so that each move costs one device submission on noisy intermediate-scale quantum (NISQ) hardware. We compile the four QUBOs onto four superconducting coupling maps and run them on three processors: IBM Heron r2, IQM Crystal 20 and IQM Star 16. The density of the conflict graph sets the compiled depth, and the depth ordering of the neighborhoods is the same on every coupling map, which matches the feasibility ordering known from annealers. On ten Taillard 20×5 instances one QAOA move lowers the mean relative deviation from the lower bound from 27.9% to between 22.1% and 25.0%. Short circuits return the same makespan on a 156-qubit and a 16-qubit device, instance by instance. Queueing stretches one move to 10–288 s, so no run reached a second move within the 10 s budget, and all quantum variants stay far behind classical neighborhood evaluation. The results give a hardware-independent rule for choosing neighborhoods and a measured cost per move.

Keywords: combinatorial optimization, QUBO, metaheuristics, simulated annealing, circuit compilation on superconducting hardware, NISQ

1 Introduction

The permutation flow shop problem (PFSP) is one of the most extensively studied NP-hard problems in production scheduling. Given n jobs that must be

* Corresponding author.

processed in the same order on m machines, the task is to find a single job permutation that minimizes the makespan $C_{\max}$, the completion time of the last job on the last machine. Because the number of permutations grows factorially, exact methods are limited to small instances, and practical solvers rely on local-search metaheuristics such as iterated local search (ILS), simulated annealing (SA), and tabu search. These methods spend the overwhelming majority of their running time in one inner operation: evaluating the moves of a neighborhood to decide where to step next. The cost of this evaluation, repeated millions of times over a search, is the dominant computational bottleneck and the natural target for acceleration.

A recent line of work offloads that evaluation to quantum hardware by casting each neighborhood as a quadratic unconstrained binary optimization (QUBO) problem and solving it on a quantum annealer [1, 2]. The annealer selects, in a single call, the subset of moves that jointly improve the makespan without conflicting, and the classical metaheuristic proceeds from the returned permutation. The gate-model paradigm, however, has remained almost untouched for scheduling, even though it offers programmable circuits, routing to arbitrary qubit pairs, and steadily growing access through cloud platforms such as IBM Quantum. Whether the same neighborhood-as-QUBO idea can be carried from the annealer to a gate-model device, and what changes when it is, is the question this paper addresses.

This paper introduces a gate-model framework that evaluates PFSP neighborhoods with the Quantum Approximate Optimization Algorithm (QAOA) [3] embedded in classical iterated local search and simulated annealing. The framework reuses the swap-selection QUBO encoding of the annealing line, so that a single problem matrix drives either an annealer or a QAOA circuit and the two paradigms are compared on identical instances. To keep the method viable on noisy intermediate-scale quantum (NISQ) hardware, we adopt a fixed-angle QAOA whose variational angles are pre-optimized offline and then frozen, which reduces the online cost to one device submission per move, the same budget the annealer spends. We formulate all four neighborhood structures used by the annealing line and analyze how the density of their conflict graphs governs circuit depth once the QUBO is compiled onto superconducting hardware, on coupling maps ranging from a heavy-hex lattice to a resonator-coupled star. Our central observation is that this depth plays the role that embedding capacity plays on the annealer: it produces the same feasibility ordering of the neighborhoods for a different, compilation-grounded reason, and windowed decomposition tames the dense neighborhoods just as it does on the annealer.

This work addresses the evaluation methodology and its analysis rather than a large-scale hardware benchmark. Current gate-model devices are noisy and shallow, so it matters to know in advance which neighborhoods compile to shallow circuits, and cloud access is metered in minutes per month, which makes the offline fixed-angle design necessary. Simulation enters only once, offline, to fix the QAOA angles; every result we report is measured on quantum processors.

2 Literature Review

The permutation flow shop problem has served as a testbed for scheduling algorithms since Taillard released the benchmark instances that remain in standard use today [4]. The problem is strongly NP-hard once three or more machines are involved, as established by Garey, Johnson, and Sethi [5], which is why practical solvers rely on heuristics. Among constructive heuristics, the method of Nawaz, Enscore, and Ham is still regarded as the strongest of its kind and is the usual starting point for local search [6]. Ruiz and Maroto compared a large collection of heuristics and metaheuristics for the makespan objective and confirmed that iterated local search and simulated annealing are among the most effective approaches [7]. These two metaheuristics, which host the quantum subroutine in our framework, trace back to this literature: simulated annealing was introduced as a general optimization method by Kirkpatrick, Gelatt, and Vecchi [8], and iterated local search was later formalized by Lourenço, Martin, and Stützle [9]. The power of such methods depends on how large a neighborhood they can search at each step. Ahuja, Ergun, Orlin, and Punnen surveyed very large-scale neighborhood search, in which a neighborhood of exponential size is explored in polynomial time [10]. Congram, Potts, and van de Velde introduced dynasearch, which uses dynamic programming to apply several independent interchange moves in a single step and gives its name to the Dynasearch neighborhood studied here [11].

A more recent direction replaces the classical evaluation of these neighborhoods with a quantum subroutine, encoding each neighborhood as a QUBO whose solution selects a set of compatible improving swaps. Bożejko, Uchroński, and Wodecki wrote the exponential Fibonacci neighborhood as a QUBO and searched it on a D-Wave annealer inside a scheduling metaheuristic [1]. A companion paper introduced the Motzkin neighborhood, whose non-crossing structure produces a sparser conflict graph than the overlap-based alternatives [12]. A third made the approach scalable through a windowed QUBO decomposition that keeps every subproblem within the connectivity of the annealer [2]. The present work adopts these QUBO encodings without change and studies their behavior on gate-model hardware. We take the QUBOs over unchanged on purpose, so that the solver is the only thing that differs between the annealing and the gate-model runs; the contribution of this paper lies in the gate-model pipeline, in the depth analysis across coupling maps, and in the measurements on three processors.

The mapping from combinatorial problems to quadratic binary form rests on the Ising model, and Lucas cataloged Ising formulations for a broad range of NP problems while bounding the number of spins each requires [13]. Quantum annealing is the physical process these formulations target; Kadowaki and Nishimori introduced it as a transverse-field method for driving an Ising system toward its ground state [14]. On real hardware, Carugno, Ferrari Dacrema, and Cremonesi evaluated job-shop scheduling on a D-Wave annealer and reported that the embedding restricts the size of the instances it can take [15].

The gate-model counterpart of quantum annealing for optimization is the quantum approximate optimization algorithm of Farhi, Goldstone, and Gut-

mann, which alternates cost and mixer layers whose angles are tuned to minimize the objective [3]; Blekos et al. review the variants that followed [16]. Zhou, Wang, Choi, Pichler, and Lukin analyzed its optimization landscape on near-term devices and found that good angles follow a regular pattern rather than scattering across the parameter space [17]. Wurtz and Lykov turned this regularity into fixed-angle conjectures that supply near-optimal parameters without instance-specific tuning [18], and Galda, Liu, Lykov, Alexeev, and Safro showed that optimal angles transfer between graphs that share local structure [19]. Sureshbabu et al. extended such parameter rules to weighted problems, where the angles are rescaled by the weights [20]; our QUBOs are weighted in the same sense. Together these results are what make the offline, fixed-angle design of our framework possible. The closest gate-model work on scheduling is that of Amaro, Rosenkranz, Fitzpatrick, Hirano, and Fiorentini, who ran several variational quantum heuristics, QAOA among them, on a small job-shop instance and found convergence on present hardware difficult [21]. Kurowski et al. applied QAOA to the job shop problem with a full-problem encoding and tested it on small instances [22], and Bożejko et al. studied hybrid quantum approximate algorithms for the flow shop problem [23]. Pelofske, Bärtschi, and Eidenbenz compared short-depth QAOA circuits with quantum annealing on the same Ising models and found the annealer ahead on current hardware [24]. Our neighborhood-level encoding keeps each circuit small, which is precisely what makes a fixed-angle approach practical where an end-to-end formulation struggles.

The hardware itself determines which circuits can run at all. Preskill characterized the noisy intermediate-scale quantum regime, in which noise sharply limits the depth of a reliable circuit [25], and Kim et al. showed that circuits of a few thousand two-qubit gates on a heavy-hex processor still give usable expectation values once error mitigation is applied [26]. IBM's superconducting processors are built on the heavy-hexagonal lattice of Chamberland, Zhu, Yoder, Hertzberg, and Cross, which reduces frequency collisions at the cost of sparse qubit connectivity [27]. Compiling a QUBO with many couplings onto such a lattice therefore requires routing, and Li, Ding, and Xie gave an efficient qubit-mapping and routing method whose inserted SWAP gates are exactly what deepen the circuits analyzed here [28]. Circuit depth on this hardware thus assumes the role that embedding capacity plays on the annealer, which is the link this paper develops.

3 Research Methodology

Our framework leaves the classical metaheuristic untouched and replaces only the neighborhood-evaluation oracle with a gate-model quantum subroutine. Every move evaluation is cast as a QUBO instance, mapped to an Ising Hamiltonian, and solved approximately by a fixed-angle QAOA circuit [3], while the QUBOs and the ILS and SA hosts are those of the annealing line [2, 12].

3.1 Swap-selection QUBO encoding

We reuse the swap-selection encoding of the annealing line [2]. The neighborhood supplies K swap candidates, and a binary variable $x_k = 1$, for $k = 1, \ldots, K$, indicates that candidate k is applied to the current permutation. Selecting a subset of compatible swaps is written as the minimization of a quadratic form

$$\min_{\mathbf{x}\in\{0,1\}^K} \mathbf{x}^{\mathsf{T}} Q\, \mathbf{x}, \tag{1}$$

where $Q_{kk} = \delta_k$, for $k = 1, \ldots, K$, is the makespan change induced by swap k, evaluated in $O(1)$ time through the Head–Tail decomposition. The matrix is upper triangular, so for $1 \le k < l \le K$ the entry Q_{kl} equals λ when candidates k and l conflict and zero otherwise, and each conflicting pair is penalized once. The penalty

$$\lambda \;=\; \sum_{k=1}^{K} |\delta_k| \;+\; 1 \tag{2}$$

dominates the objective, so any solution that selects two conflicting swaps is more expensive than any feasible one. The one-hot Adjacent encoding is the exception. There every pair conflicts, so the penalty also enters the diagonal, with $Q_{kk} = \delta_k - P$ and $Q_{kl} = 2P$ for $k < l$, where $P = 2\max_k|\delta_k| + 1$. The pattern of penalized pairs is the fingerprint of the neighborhood (Section 3.5).

3.2 From the QUBO to a QAOA circuit

The change of variables $Z_k = 2x_k - 1$, i.e. $x_k = (1 + Z_k)/2$, turns the QUBO (1) into an Ising cost Hamiltonian [13]

$$H_C \;=\; \sum_{i=1}^{K} h_i Z_i \;+\; \sum_{1\le i<j\le K} J_{ij} Z_i Z_j \;+\; \text{const}, \tag{3}$$

where Z_i and X_i are the Pauli-Z and Pauli-X operators acting on qubit i, with one qubit per swap candidate, and the linear fields h_i and the couplings J_{ij} follow directly from the entries of Q. A depth-p QAOA prepares the parameterized state

$$|\gamma,\beta\rangle \;=\; \Big(\textstyle\prod_{l=1}^{p} e^{-i\beta_l H_M}\, e^{-i\gamma_l H_C}\Big)|+\rangle^{\otimes K}, \qquad H_M = \textstyle\sum_{i=1}^{K} X_i, \tag{4}$$

with the transverse-field mixer H_M. The circuit carries p layers and two angles per layer, γ_l for the cost evolution and β_l for the mixer, which we optimize over $\gamma_l \in [0, \pi]$ and $\beta_l \in [0, \pi/2]$. For $p = 1$ each layer compiles to elementary gates. An initial layer of Hadamard gates places the register in an equal superposition over all move selections. The cost evolution $e^{-i\gamma H_C}$ is realized as a single-qubit rotation $R_Z(2\gamma h_i)$ on every variable together with a two-qubit rotation $R_{ZZ}(2\gamma J_{ij})$ on every coupled pair, and the mixer $e^{-i\beta H_M}$ as a rotation $R_X(2\beta)$ on every qubit. Sampling the circuit gives a set of bitstrings, each a candidate swap set; Section 3.4 explains how one of them becomes the move. Figure 1 shows the resulting $p = 1$ circuit for a small Fibonacci instance. The set of R_{ZZ} gates is exactly the edge set of the conflict graph of Q, so the neighborhood structure alone determines the two-qubit layer of the circuit.

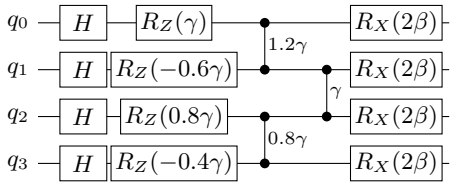


Fig. 1. QAOA circuit ($p = 1$) for a four-variable Fibonacci neighborhood.

3.3 Fixed-angle QAOA with offline angle optimization

Variational QAOA re-optimizes the angles (γ, β) for every instance, spending tens to hundreds of circuit executions before a single move is chosen. Under NISQ noise and a tight quantum-runtime budget—the IBM Open plan grants roughly ten minutes of quantum processing unit (QPU) time per twenty-eight-day window—this is not viable inside a metaheuristic that evaluates thousands of moves. We therefore fix the angles per neighborhood type and obtain them offline, once. For each of the four neighborhoods a single angle pair $(\gamma^\star, \beta^\star)$ is optimized on a classical statevector simulation by an exhaustive grid of 25×13 points over that range, by minimizing the expected cost

$$(\gamma^\star, \beta^\star) \;=\; \arg\min_{\gamma,\beta} \langle \gamma, \beta | H_C | \gamma, \beta \rangle, \qquad \gamma \in [0, \pi],\ \beta \in [0, \pi/2], \tag{5}$$

averaged over a training set of windowed QUBOs of that neighborhood. The set comes from three Taillard instances (tai20_5 nos. 0 and 1, tai20_10 no. 0), each taken at the identity permutation and cut into windows of six positions with step three, which gives 21 QUBOs for Adjacent and Fibonacci and 11 for Dynasearch and Motzkin. Two of the training instances are also among the ten test instances, and because every run starts from the identity, their first moves meet the training windows; at $p = 1$ the optimal angles are nearly constant across instances, so we do not expect this to bias the results. The resulting angles $(\gamma^\star, \beta^\star)$ are $(19\pi/24, 7\pi/24)$ for Adjacent, $(21\pi/24, 5\pi/24)$ for Fibonacci, $(17\pi/24, \pi/12)$ for Dynasearch and $(17\pi/24, \pi/8)$ for Motzkin. The angles are then frozen and reused for every move of that neighborhood at run time, so the offline cost is amortized over the whole search and the online cost per move stays at a single submission. This strategy relies on angle transferability: for a fixed neighborhood the optimal $p = 1$ angles are near-constant across instances, a property this design relies on, and which the fixed-angle conjectures of Wurtz and Lykov [18] support for regular graphs. On hardware each move therefore costs one submission, the direct gate-model analog of the annealer's single call per move, which is the basis for a fair wall-clock comparison. For Adjacent and Fibonacci that submission holds a single circuit; for the windowed neighborhoods it holds one circuit per window. We work throughout at $p = 1$, the depth that current NISQ hardware supports reliably. Whether greater depths pay off on hardware is a question for future work: it requires many more circuit executions

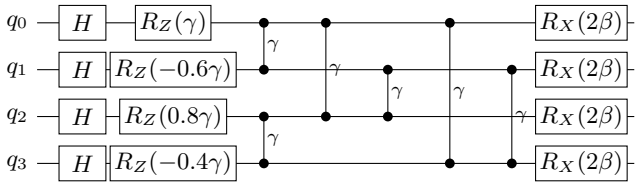
than a queued, quota-limited account allows, and therefore better access to the device than we had here.

3.4 From a sample to a move

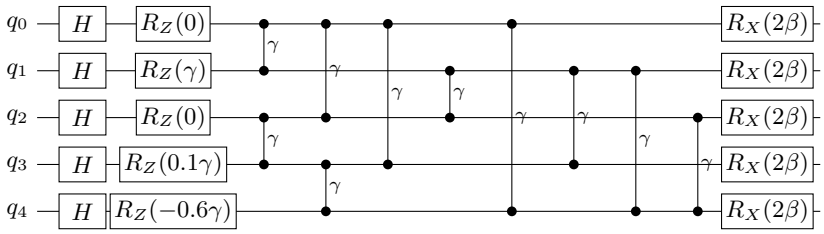
Each circuit is sampled with 4096 shots, and we keep the sampled bitstring of lowest energy under the original QUBO (1). The single most frequent bitstring of a $p = 1$ state is usually the all-zeros assignment or a high-energy state with many ones, so it is a poor readout. The bits equal to one name the selected swap candidates. For the dense neighborhoods the permutation is cut into windows of six positions that overlap by half, all windows of a move are sent as one job, and the swaps selected in all windows are joined into one list. A greedy pass from left to right then restores feasibility: a Dynasearch swap (i, j) is kept if it does not overlap the last kept swap, a Motzkin swap is kept if it neither crosses a kept swap nor shares an endpoint with it, and Fibonacci keeps adjacent swaps that do not share a position. The same pass drops a swap that two overlapping windows both selected. For Adjacent each window proposes at most one swap and the move takes the proposal with the smallest δ_k, which is the minimizer of the global one-hot QUBO. The kept swaps touch disjoint positions, so they are applied at once, and the makespan of the new permutation is then computed from scratch in $O(nm)$ time rather than from the sum of the δ_k. If no swap survives the repair, the method falls back to the best single improving swap; no move reported in Section 4 came from this fallback, since every reported Fibonacci, Dynasearch and Motzkin move changes four or more positions.

3.5 Neighborhoods and their conflict graphs

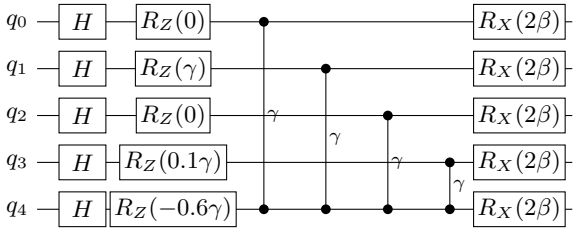
All four neighborhoods reuse their annealing QUBOs [2, 12] and differ only in the coupling pattern of the R_{ZZ} layer in (4), that is, in the topology of the conflict graph. The Adjacent neighborhood selects one of the $n - 1$ adjacent transpositions under a one-hot constraint, so every pair of variables conflicts and its conflict graph is complete. The Fibonacci neighborhood penalizes only consecutive overlapping swaps and so yields a tridiagonal chain, the sparsest of the four. The Dynasearch and Motzkin neighborhoods both act on endpoint-swap intervals but apply different conflict rules: Dynasearch penalizes every pair of overlapping intervals, nested ones included, whereas Motzkin admits nested intervals and penalizes only crossings and shared endpoints. Motzkin's conflict graph is therefore a subgraph of Dynasearch's and is strictly sparser, which is precisely the property that motivates the Motzkin neighborhood [12]. The Adjacent and Dynasearch graphs are dense and compile to the deepest circuits, while Fibonacci's tridiagonal chain stays shallowest; Motzkin sits between them, sparser than Dynasearch but far denser than Fibonacci. Figure 2 shows example $p = 1$ circuits for the Adjacent, Dynasearch, and Motzkin neighborhoods; the Fibonacci circuit is the one in Fig. 1. Each R_{ZZ} gate corresponds to one edge of the conflict graph, so the two-qubit gate count is read directly off the graph.



(a) Adjacent: complete conflict graph (dense)



(b) Dynasearch: all interval overlaps penalized (dense)



(c) Motzkin: nesting admitted, only crossings penalized (sparse)

Fig. 2. Example QAOA cost-and-mixer layers ($p = 1$) for the Adjacent, Dynasearch, and Motzkin neighborhoods; Fibonacci is in Fig. 1.

3.6 Compilation across coupling maps

An R_{ZZ} gate requires its two qubits to be physically adjacent, so on any fixed coupling map the compiler routes the remaining pairs through SWAP networks, which deepen the circuit and inject additional noise. Circuit depth after transpilation therefore plays the role that embedding capacity plays on the annealer. To separate the effect of the coupling map from everything else, we compiled the same four swap-selection QUBOs, taken from the first Taillard 20×5 instance, onto four processors: the heavy-hex lattice of IBM Heron r2, the square lattices of IQM Crystal 20 and Crystal 54, and the star architecture of IQM Star 16, where every qubit couples to one central resonator and a qubit-resonator MOVE operation takes the place of a SWAP network. All figures are taken at $p = 1$, with windows of six for the dense neighborhoods, and summed over the circuits of one move; on Star 16 the two-qubit count is controlled-Z (CZ) plus MOVE. Transpilation consumes no device time, so this comparison cost no quota.

Table 1 admits two readings. The ordering of the four neighborhoods by compiled depth is the same on every coupling map, with Fibonacci well below

Adjacent and both well below Motzkin and Dynasearch, so the feasibility ordering the annealing line observed does not depend on the choice of hardware. The absolute depths do depend on it, and heavy-hex is the least favourable of the four, since the square lattices compile the same circuits between 1.7 and 2.6 times shallower.

The Fibonacci row is the control. Its tridiagonal QUBO compiles to fourteen CZ gates on all four maps, because a tridiagonal interaction graph embeds in a path and needs no routing anywhere, so nothing in that row can be attributed to the device. The star architecture removes routing for the dense neighborhoods too, cutting the CZ count of Dynasearch from 1668 to 696, but it pays in MOVE operations that serialize through the single resonator, which is why its depth lands between the two lattices rather than below them. Connectivity buys gate count, not depth.

Table 1. Compiled depth d and two-qubit gate count for one move of each neighborhood, with the mean over the four coupling maps on the right.

	Heron r2		Crystal 20		Crystal 54		Star 16		*Average*	
Neighborhood	d	2q	d	2q	d	2q	d	2q	d	2q
Fibonacci	58	14	**22**	14	**22**	14	38	28	*35*	*18*
Adjacent	337	195	200	153	**188**	159	227	**180**	*238*	*172*
Motzkin	2208	1455	1280	1080	**1177**	1077	1438	**1180**	*1526*	*1198*
Dynasearch	2509	1668	1466	1287	**1383**	1275	1758	**1438**	*1779*	*1417*

3.7 Hardware and software

The circuits target the IBM Heron r2 processor `ibm_fez` (156 qubits, heavy-hex coupling map with 352 couplings, native gate set CZ, RZ, SX, X), accessed through Qiskit Runtime (`qiskit` 2.5.1 with `qiskit-ibm-runtime`). A second set of measurements runs on two IQM processors reached through IQM Resonance: `garnet` (Crystal 20, 20 qubits on a square lattice with 30 couplings) and `sirius` (Star 16, 16 qubits coupled to one central resonator). Both use a CZ two-qubit gate with single-qubit rotations, and `sirius` adds the qubit-resonator MOVE operation, so the circuits of Section 3 compile to all three devices unchanged. The angle schedule is fixed beforehand by the offline calibration of Section 3.3, so that scarce QPU time is spent on measurement rather than on angle tuning. Every circuit is sampled with 4096 shots at $p = 1$. On `ibm_fez` the circuits are transpiled with the Qiskit preset pass manager at optimization level 1, which chooses the qubit layout for each job, and executed through the Qiskit Runtime `SamplerV2` primitive with default options, that is, without error mitigation, dynamical decoupling or twirling. On the IQM devices the circuits are transpiled at optimization level 1 and executed without error mitigation.

The software is Python 3.11, `qiskit` 2.5.1 and `qiskit-ibm-runtime` 0.48.0 for IBM, and `qiskit` 2.1.2 with `iqm-client` 35.0.3 for IQM.

4 Results and Discussion

The evaluation has two parts. The first repeats the protocol of the annealing line on a heavy-hex processor, which places the gate rows beside the published classical and annealing measurements on the same instances. The second changes the coupling map and holds everything else fixed, which puts the compilation argument of Section 3.6 to a test on hardware rather than in the compiler alone.

4.1 Evaluation protocol

We follow the protocol of the windowed annealing study [2] so that the two sets of results are directly comparable: Taillard PFSP instances with $n = 20$ jobs, ten instances per configuration, and the same lower bounds. Within that protocol the quota reached one machine count, so every measurement we report is on the ten $m = 5$ instances; the $m = 10$ and $m = 20$ configurations belong to the full protocol discussed in Section 5. Each neighborhood drives an iterated-local-search and a simulated-annealing loop; at every move the swap-selection QUBO of Section 3 is assembled and handed to the fixed-angle QAOA circuit in place of the annealer. Solution quality is reported as the percentage relative deviation $\mathrm{PRD} = (C_{\max} - \mathrm{LB})/\mathrm{LB} \times 100$ from the Taillard lower bound LB, averaged over instances under a fixed wall-clock budget. Every run starts from the identity permutation, and the reported PRD belongs to the best makespan the run has seen. Each move is read out as described in Section 3.4.

4.2 Execution on the heavy-hex processor

We ran the four gate neighborhoods on `ibm_fez` over the ten Taillard 20×5 instances under both metaheuristics, eighty runs in total, consuming about 460 s of QPU time. All windows of a move are submitted as one job, so a move costs a single submission irrespective of how many windows the decomposition produces.

The runs were started with the longest budget of the protocol, 10 s, and every one of the eighty runs completed exactly one move and overshot that budget: measured run times ranged from 10 to 288 s. The overshoot is dominated by queueing and job overhead rather than by computation, since the billed QPU time per move is about 3 s for the linear neighborhoods and 7 s for the windowed ones. A run performs at least one move, so no shorter budget could admit a second move either. Table 2 therefore reports a single value per neighborhood, the PRD after one move, beside the PRD of the identity start and the median wall-clock time of the move.

Adjacent returns 25.04 under both metaheuristics because both loops start from the identity permutation, and the choice of metaheuristic cannot matter before a second move. For the other three neighborhoods the ILS and SA means

differ by less than half a percentage point, while single instances differ by up to 5.8 points. With one move from the same start the two loops perform the same computation, so these differences come from sampling on the device. The standard deviation in Table 2 measures how much the instances differ, not measurement noise; compared instance by instance, Dynasearch beats Adjacent on eight of the ten instances, Fibonacci and Motzkin on seven, while the three multi-swap neighborhoods cannot be separated from one another at this sample size.

The same circuits run on two further processors, with different coupling maps, in Section 4.4.

4.3 Comparison with the annealing back-ends

Table 3 places the gate rows beside the classical and annealing measurements of the windowed study on the same instances; the classical and annealing columns are taken at the 10 s budget, and the gate columns after one move, which exceeded it.

On Dynasearch the gate processor returns the lowest PRD of the three quantum back-ends, 22.12 against 25.85 for the annealer and 26.24 for its windowed variant, and it does so under both metaheuristics. Motzkin places it between them: better than the windowed annealer, which is the variant it should be compared with since both decompose the neighborhood into windows, and worse than the full annealing formulation. On the two linear neighborhoods the gate rows are the weakest of the three, by about five points.

Part of that gap is access latency rather than solution quality. An annealer call costs on the order of 1 s, so from 2 s onward the annealing runs convert the remaining budget into further moves, whereas a gate move costs about 30 s and no budget in the protocol admits a second one. At the budgets up to 1 s, where both back-ends answer after a single call, the annealing and gate rows for the linear neighborhoods sit within a few tenths of a point of each other in the annealing study; the comparison in Table 3 is taken at the budget least favorable to the gate device.

All quantum back-ends remain far from the classical baselines at this instance size, as they did in the annealing study.

Table 2. Mean PRD [%] ± standard deviation over the ten instances on `ibm_fez`: identity start and PRD after one move, with the median wall-clock time of the move.

		After one move		
Neighborhood	Start	ILS	SA	Time per move [s]
Gate Adjacent	27.90 ± 9.8	25.04 ± 10.0	25.04 ± 10.0	25.1
Gate Fibonacci	27.90 ± 9.8	23.41 ± 9.2	23.48 ± 9.0	16.4
Gate Dynasearch	27.90 ± 9.8	$\mathbf{22.12} \pm 8.0$	$\mathbf{22.38} \pm 8.7$	33.2
Gate Motzkin	27.90 ± 9.8	23.12 ± 8.0	23.59 ± 8.1	29.9

Table 3. PRD [%] on Taillard 20×5, ten instances, classical and annealing columns at $t_{\max} = 10$ s; gate QAOA† after one move, which took 10–288 s. Bold marks the best quantum back-end per neighborhood.

	Classical		Annealer		Annealer, win.		Gate QAOA†	
Neighborhood	ILS	SA	ILS	SA	ILS	SA	ILS	SA
Adjacent	3.41	6.98	20.17	20.18	**20.03**	**20.16**	25.04	25.04
Fibonacci	3.82	9.24	**17.94**	**17.82**	18.20	18.12	23.41	23.48
Dynasearch	2.68	3.36	25.85	26.39	26.24	26.21	**22.12**	**22.38**
Motzkin	2.64	3.25	**19.02**	**19.10**	25.22	25.15	23.12	23.59
Average	3.14	5.71	20.75	20.87	22.42	22.41	23.42	23.62

4.4 The same neighborhoods on three coupling maps

The depth argument of Section 3.6 concerns compilation, so it can be tested by changing the coupling map and holding everything else fixed. We repeated the protocol under iterated local search on `garnet` and on `sirius` with the same QUBOs, angles, shots and identity start. For Adjacent and Fibonacci, whose moves compile to a single circuit each, the quota covered the ten-instance protocol. The result is an equality rather than an agreement. On all nine instances that fit the sixteen qubits of `sirius`, Adjacent returns the same makespan there as on `ibm_fez`, instance by instance, and both columns read 25.79 ± 10.26; the gap to Table 2 is the omitted instance, not the device. Fibonacci agrees on six of those nine and differs by at most 0.85 percentage points on the rest, at 24.39 ± 8.99 against 24.43 ± 9.15. A tridiagonal or near-tridiagonal conflict graph compiles to a circuit short enough that a heavy-hex lattice of 156 qubits and a 16-qubit resonator device return the same answer, and the Fibonacci row of Table 1 says why: both compile it to fourteen CZ gates.

The dense neighborhoods behave differently, and Table 4 reports them on the first instance, which is as far as the quota reached. The runs start from the identity permutation, whose makespan there is 1448 against a lower bound of 1232, a PRD of 17.53, and the Heron r2 column is recomputed from that one instance rather than taken from the ten-instance mean. Motzkin improves monotonically as the coupling map changes, by 3.65 points in total, while Dynasearch does not follow that order and is worst on the square lattice. One instance cannot separate a device effect from sampling noise, and we report the row as measured.

Taken together the two groups describe a threshold rather than a gradient. Adjacent (195, 153 and 90 CZ gates on the three maps) returns the same makespan everywhere, while Motzkin (1455, 1080 and 576) drops from 16.96 to 13.31: below some compiled depth the coupling map is invisible to the result, and above it the map decides.

The sixteen qubits of `sirius` are also a binding constraint even at $n = 20$, because the width of the swap-selection QUBO varies between 5 and 18 variables across the ten instances and one of them does not fit.

Table 4. PRD [%] after one move on the first Taillard 20×5 instance. Bold marks the best coupling map; the Adjacent row is unmarked because the three agree exactly.

Neighborhood	Heron r2	Crystal 20	Star 16
Adjacent	12.82	12.82	12.82
Fibonacci	12.26	**12.01**	**12.01**
Dynasearch	12.42	15.10	**12.09**
Motzkin	16.96	14.85	**13.31**
Average	13.62	13.70	12.56

5 Conclusions

We have carried the neighborhood-as-QUBO idea from the quantum annealer to the gate model, with one shared QUBO per neighborhood and angles fixed offline. The analysis identifies compiled circuit depth as the gate-model counterpart of embedding capacity: a conflict graph that exceeds an annealer's connectivity also routes into a deep, noisy circuit, so the two paradigms order the neighborhoods for feasibility in the same way for otherwise unrelated reasons. Compiling the same four QUBOs onto a heavy-hex lattice, two square lattices and a resonator-coupled device leaves that ordering unchanged while moving the absolute depths by a factor of two, which makes the argument a statement about compilation rather than about one vendor's topology, and heavy-hex is the least favourable of the four. On hardware the consequence is a threshold rather than a trend: the neighborhoods that compile to short circuits return the same makespan on a 156-qubit heavy-hex processor and on a 16-qubit resonator device, instance by instance.

What we could measure is bounded by allocation rather than by method. Every run completed a single move irrespective of its budget, because a move costs about 30 s of wall-clock time against 3–7 s of computation, and the same move takes 12 s on a less heavily shared processor, so that figure measures contention; the time dimension of the protocol therefore collapses. The full protocol of the annealing line, ten instances at five seeds across six budgets for both metaheuristics, costs about 550 credits of device time per processor at the rates we measured, against the thirty a monthly allocation provides, and we intend to run it once a larger allocation is in place, on the same instances, seeds and budgets and on each of the coupling maps. Scaling in n carries a cost of its own, since billing tracks the number of circuits rather than their depth: one move of a dense neighborhood needs six circuits at $n = 20$ and 166 at $n = 500$, so the price grows linearly with instance size where one annealer call covers the whole QUBO up to its embedding limit. One question we leave open is methodological, because a strict wall-clock criterion charges the queue wait to the algorithm although that wait varies by an order of magnitude between identical submissions.

Data and code availability. The code of the framework, the calibrated angles and the measured results are available at https://github.com/RemigiuszWoj/PhD and archived at https://doi.org/10.5281/zenodo.24106982. The raw output of the four Crystal 20 runs in Table 4 was lost, and the values reported there are taken from the laboratory notes.

Disclosure of AI use. The authors used a large language model as a writing and code-review aid in preparing this manuscript; the results, their interpretation and the final text are the authors' own.

References

1. Bożejko, W., Uchroński, M., Wodecki, M.: The new exponential Fibonacci neighborhood as QUBO generated on a D-Wave quantum computing environment for the permutational scheduling problem. Sci. Rep. (2026). https://doi.org/10.1038/s41598-026-66284-9
2. Wojewódzki, R., Bożejko, W.: Scalable quantum neighborhood search for the permutation flow shop problem via windowed QUBO decomposition. SSRN preprint 7411334 (2026). https://doi.org/10.2139/ssrn.7411334
3. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. arXiv:1411.4028 (2014). https://doi.org/10.48550/arXiv.1411.4028
4. Taillard, É.: Benchmarks for basic scheduling problems. Eur. J. Oper. Res. **64**(2), 278–285 (1993). https://doi.org/10.1016/0377-2217(93)90182-M
5. Garey, M.R., Johnson, D.S., Sethi, R.: The complexity of flowshop and jobshop scheduling. Math. Oper. Res. **1**(2), 117–129 (1976). https://doi.org/10.1287/moor.1.2.117
6. Nawaz, M., Enscore, E.E., Ham, I.: A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. Omega **11**(1), 91–95 (1983). https://doi.org/10.1016/0305-0483(83)90088-9
7. Ruiz, R., Maroto, C.: A comprehensive review and evaluation of permutation flowshop heuristics. Eur. J. Oper. Res. **165**(2), 479–494 (2005). https://doi.org/10.1016/j.ejor.2004.04.017
8. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983). https://doi.org/10.1126/science.220.4598.671
9. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search. In: Glover, F., Kochenberger, G.A. (eds.) Handbook of Metaheuristics, pp. 320–353. Kluwer, Boston (2003). https://doi.org/10.1007/0-306-48056-5_11
10. Ahuja, R.K., Ergun, Ö., Orlin, J.B., Punnen, A.P.: A survey of very large-scale neighborhood search techniques. Discrete Appl. Math. **123**(1–3), 75–102 (2002). https://doi.org/10.1016/S0166-218X(01)00338-9
11. Congram, R.K., Potts, C.N., van de Velde, S.L.: An iterated dynasearch algorithm for the single-machine total weighted tardiness scheduling problem. INFORMS J. Comput. **14**(1), 52–67 (2002). https://doi.org/10.1287/ijoc.14.1.52.7712
12. Wojewódzki, R., Bożejko, W.: Motzkin neighborhood evaluation in iterated local search and simulated annealing for the permutational flow shop problem. In: SOCO 2026, CCIS, vol. 3046, pp. 144–155. Springer (2026). https://doi.org/10.1007/978-3-032-29254-4_12

13. Lucas, A.: Ising formulations of many NP problems. Front. Phys. **2**, 5 (2014). https://doi.org/10.3389/fphy.2014.00005
14. Kadowaki, T., Nishimori, H.: Quantum annealing in the transverse Ising model. Phys. Rev. E **58**(5), 5355–5363 (1998). https://doi.org/10.1103/PhysRevE.58.5355
15. Carugno, C., Ferrari Dacrema, M., Cremonesi, P.: Evaluating the job shop scheduling problem on a D-Wave quantum annealer. Sci. Rep. **12**, 6539 (2022). https://doi.org/10.1038/s41598-022-10169-0
16. Blekos, K., Brand, D., Ceschini, A., Chou, C.-H., Li, R.-H., Pandya, K., Summer, A.: A review on Quantum Approximate Optimization Algorithm and its variants. Phys. Rep. **1068**, 1–66 (2024). https://doi.org/10.1016/j.physrep.2024.03.002
17. Zhou, L., Wang, S.-T., Choi, S., Pichler, H., Lukin, M.D.: Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices. Phys. Rev. X **10**, 021067 (2020). https://doi.org/10.1103/PhysRevX.10.021067
18. Wurtz, J., Lykov, D.: Fixed-angle conjectures for the quantum approximate optimization algorithm on regular MaxCut graphs. Phys. Rev. A **104**, 052419 (2021). https://doi.org/10.1103/PhysRevA.104.052419
19. Galda, A., Liu, X., Lykov, D., Alexeev, Y., Safro, I.: Transferability of optimal QAOA parameters between random graphs. In: 2021 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 171–180. IEEE (2021). https://doi.org/10.1109/QCE52317.2021.00034
20. Sureshbabu, S.H., Herman, D., Shaydulin, R., Basso, J., Chakrabarti, S., Sun, Y., Pistoia, M.: Parameter setting in quantum approximate optimization of weighted problems. Quantum **8**, 1231 (2024). https://doi.org/10.22331/q-2024-01-18-1231
21. Amaro, D., Rosenkranz, M., Fitzpatrick, N., Hirano, K., Fiorentini, M.: A case study of variational quantum algorithms for a job shop scheduling problem. EPJ Quantum Technol. **9**, 5 (2022). https://doi.org/10.1140/epjqt/s40507-022-00123-4
22. Kurowski, K., Pecyna, T., Slysz, M., Różycki, R., Waligóra, G., Węglarz, J.: Application of quantum approximate optimization algorithm to job shop scheduling problem. Eur. J. Oper. Res. **310**(2), 518–528 (2023). https://doi.org/10.1016/j.ejor.2023.03.013
23. Bożejko, W., Idzikowski, R., Jagielski, D., Klempous, R., Trotskyi, S., Tykierko, M., Uchroński, M., Wodecki, M.: Hybrid quantum approximate algorithms for solving flow shop scheduling problem. In: Proc. IEEE 24th Int. Symp. CINTI, pp. 117–122. IEEE (2024). https://doi.org/10.1109/CINTI63048.2024.10830818
24. Pelofske, E., Bärtschi, A., Eidenbenz, S.: Short-depth QAOA circuits and quantum annealing on higher-order Ising models. npj Quantum Inf. **10**, 30 (2024). https://doi.org/10.1038/s41534-024-00825-w
25. Preskill, J.: Quantum computing in the NISQ era and beyond. Quantum **2**, 79 (2018). https://doi.org/10.22331/q-2018-08-06-79
26. Kim, Y., Eddins, A., Anand, S., Wei, K.X., van den Berg, E., Rosenblatt, S., Nayfeh, H., Wu, Y., Zaletel, M., Temme, K., Kandala, A.: Evidence for the utility of quantum computing before fault tolerance. Nature **618**, 500–505 (2023). https://doi.org/10.1038/s41586-023-06096-3
27. Chamberland, C., Zhu, G., Yoder, T.J., Hertzberg, J.B., Cross, A.W.: Topological and subsystem codes on low-degree graphs with flag qubits. Phys. Rev. X **10**, 011022 (2020). https://doi.org/10.1103/PhysRevX.10.011022
28. Li, G., Ding, Y., Xie, Y.: Tackling the qubit mapping problem for NISQ-era quantum devices. In: Proc. 24th Int. Conf. ASPLOS, pp. 1001–1014. ACM (2019). https://doi.org/10.1145/3297858.3304023